React vs JS — Event Propagation

Same model, different machinery

Is React's event propagation different from native JavaScript?

Almost the same conceptually, with one structural twist. React mirrors the DOM event model — same 3 phases, same API for stopping events — but runs the propagation through the **JSX tree** using a single **delegated listener** at the root container, not real listeners on every element.

> ■ **The 1-line takeaway**
>
> React's propagation looks identical to native DOM at the API level (capture, target, bubble). Under the hood it's a single root-level listener walking the JSX/fiber tree. **stopPropagation** only affects React's synthetic system — the native event still bubbles through the real DOM.

What's the same

React's event propagation conceptually mirrors the DOM model:

- Same 3 phases: **capture** → **target** → **bubble**
- Same default phase: **bubble**
- Same API: `e.stopPropagation()`, `e.preventDefault()`, `e.stopImmediatePropagation()`
- Same `e.target` vs `e.currentTarget` distinction
- Capture phase opt-in: append **Capture** (e.g. `onClickCapture`)

So if you understand DOM events, you mostly understand React events.

What's different

1. React events propagate through the JSX tree, not the real DOM

React events bubble through the **component hierarchy in JSX**, based on the fiber tree React maintains. This usually matches the DOM, but diverges when components render through a **portal**.

```
function Modal({ children }) {
return createPortal(children, document.body); // mounted at body in DOM
}

&lt;div onClick={() =&gt; console.log('JSX parent')}&gt;
&lt;Modal&gt;
&lt;button onClick={() =&gt; console.log('button')}&gt;Click&lt;/button&gt;
&lt;/Modal&gt;
&lt;/div&gt;

// Click button -&gt; output:
// button
// JSX parent ← React bubbles to JSX parent, even though
// DOM-wise the button is mounted in &lt;body&gt;
```

In raw DOM, the event would bubble to <body>, not to that JSX parent . React deliberately follows the JSX tree — making portals composable without breaking event flow.

2. React uses delegation under the hood

When you write 1000 `onClick`s, React does **not** attach 1000 real DOM listeners. It attaches **one** listener per event type at the root container, and walks the fiber tree on every event to invoke matching handlers.

```
You write: React adds:
1000 &lt;button onClick={fn} /&gt; 1 real 'click' listener at the root container

On click → React walks the fiber tree and calls every matching onClick in bubble order
```

> ■ **Historical note**
>
> React 16 attached listeners at `document`. **React 17+** moved them to the **root container** (`<div id="root">`). This was a breaking change — it's why some `stopPropagation` behavior changed subtly between v16 and v17.

3. stopPropagation only affects React's tree

A native `document` listener can still fire even after React's `stopPropagation`. Because React's listener sits at the root container, the native event keeps bubbling up the real DOM after React is done.

```
useEffect(() =&gt; {
document.addEventListener('click', () =&gt; console.log('native'));
}, []);

return (
&lt;button onClick={(e) =&gt; {
e.stopPropagation(); // stops React tree only
console.log('react');
}}&gt;Click&lt;/button&gt;
);

// Output:
// react
// native ← still fires!
```

To stop the native event too, drop into the underlying browser event:

```
onClick={(e) =&gt; {
e.nativeEvent.stopImmediatePropagation();
}}
```

4. React synthesizes bubbling for onFocus / onBlur

Native `focus` and `blur` **do not bubble**. React synthesizes bubbling for them so you can listen on any parent:

```
&lt;form onFocus={() =&gt; console.log('focus inside')}&gt;
&lt;input /&gt;
&lt;input /&gt;
&lt;/form&gt;

// Tabbing into ANY inner input fires the form's onFocus.
// Native 'focus' wouldn't bubble to the form — you'd need 'focusin'.
```

5. Synthetic events are wrappers

e in a React handler is a **SyntheticEvent**, not the native browser event. Same API, but cross-browser normalized. The real browser event lives on `e.nativeEvent`.

```
&lt;button onClick={(e) =&gt; {
console.log(e); // SyntheticEvent (React's wrapper)
console.log(e.nativeEvent); // MouseEvent (native, browser's original)
}}&gt;Click&lt;/button&gt;
```

> ■ **React 16 vs 17+**
>
> React 16 had **event pooling** — synthetic events were reused for performance, so you couldn't access e asynchronously without `e.persist()`. React 17+ removed pooling — you can use e freely now.

Quick comparison table

Aspect	DOM (native)	React
Phases	capture → target → bubble	same
Default phase	bubble	bubble
Capture syntax	`addEventListener(_, _, true)`	`onClickCapture`
Stop method	`e.stopPropagation()`	`e.stopPropagation()` (synthetic only)
Real DOM listeners	one per `addEventListener`	one delegated listener at root, per event type
Propagates through	real DOM tree	JSX (fiber) tree — matters with portals
focus / blur bubble?	No	Yes (synthesized)
Event object	MouseEvent, KeyboardEvent, etc.	SyntheticEvent (real one at `e.nativeEvent`)

Mental model — two parallel worlds

```
REACT WORLD NATIVE / DOM WORLD
■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■
SyntheticEvent ←wraps→ MouseEvent / KeyboardEvent / ...
JSX tree real DOM tree
onClick / onClickCapture addEventListener
e.stopPropagation() e.stopPropagation() (real)

React stops here. ■
■
▼ the native event is still bubbling
up the real DOM, headed for document and window

To stop the native event too:
e.nativeEvent.stopImmediatePropagation();
```

Interview Q&A

■ Is React's event propagation different from native JavaScript?

Conceptually it's the same — same 3 phases (capture, target, bubble), same default phase (bubble), same API for `stopPropagation` and `preventDefault`. Structurally it's different: React uses one delegated listener at the root container per event type, propagates through the JSX tree (not the DOM tree, which matters with portals), wraps events in SyntheticEvent, and synthesizes bubbling for focus/blur.

■ What is a SyntheticEvent?

React's cross-browser wrapper around the native browser event. Same API (`target`, `preventDefault`, `stopPropagation`, etc.) but normalized across browsers. The underlying native event is accessible via `e.nativeEvent`.

■ Why does e.stopPropagation in React not stop a native document listener?

Since React 17, React's synthetic listener sits at the root container, not document. By the time React calls your handler, the native event has already bubbled up through the real DOM. `e.stopPropagation()` halts React's synthetic walk, but the native event continues bubbling to document. Use `e.nativeEvent.stopImmediatePropagation()` to stop the native event too.

■ What changed in React 17 about event handling?

Two big things: (1) Listeners moved from `document` to the root container — better for apps with multiple React roots and micro-frontends. (2) Event pooling was removed — you can now use e asynchronously without calling `e.persist()`.

■ How does React event propagation behave with portals?

React events bubble through the **JSX tree**, not the real DOM. So even if a modal is rendered to document.body via createPortal, clicks inside still bubble to the JSX parent that contained the Modal component. This is intentional — keeps portals composable without breaking event-handling expectations.

■ Why does React synthesize bubbling for focus and blur?

Native focus/blur don't bubble — you'd need focusin/focusout for delegation. React abstracts this away by making onFocus/onBlur bubble through the JSX tree, so you can listen on parent components naturally without thinking about which DOM events bubble and which don't.

TL;DR

■ **Same model, different machinery**

Same as DOM:

• 3 phases: capture → target → bubble
• Default phase: bubble
• Same API: stopPropagation, preventDefault, target, currentTarget
• Capture phase = append **Capture** to the prop (onClickCapture)

Different from DOM:

• Propagates through the JSX/fiber tree (matters with portals)
• ONE delegated listener per event type at the root container
• `e` is a SyntheticEvent (native lives at `e.nativeEvent`)
• `e.stopPropagation()` only affects React's synthetic system
• React synthesizes bubbling for onFocus/onBlur (native focus/blur don't bubble)

One-line interview answer

■ **If you only remember one sentence...**

"React mirrors DOM event propagation (capture → target → bubble) but runs it through the JSX tree using a single delegated listener at the root container, with SyntheticEvent wrappers around native events. ***stopPropagation*** *only stops React's synthetic propagation — the native event keeps bubbling unless you also call* ***e.nativeEvent.stopImmediatePropagation()****."*

Reference for interview prep — Akshay Chavhan